

FunC v0.5.0

Ergonomic and Strongly Typed Design Proposal

Tomi Setsu

April 18, 2026

Document version: v3 (Draft) | **Supersedes:** v1 (initial proposal), v2
(10 Canonical Decisions locked)

Abstract

This document proposes a new source-level design target for FunC, version 0.5.0. It starts from the currently implemented FunC 0.4.6 surface documented in `doc/func.tex`, but reworks the language around five principles:

1. strong domain types for TVM/TOS values such as typed cells, `Slice`, `Builder`, `Address`, and typed messages;
2. algebraic data types and exhaustive pattern matching;
3. explicit fallibility through `Option` and `Result`;
4. explicit mutability and explicit side effects;
5. module-, trait-, and generic-oriented reuse suitable for a standard library and contract ecosystem.

The language direction intentionally moves toward Rust-like ergonomics, but does not attempt to become “Rust on TVM”. In particular, it does not introduce ownership, borrowing, lifetimes, macro metaprogramming, or a synchronous call model that would conflict with TOS’s asynchronous, actor-like execution.

This is a design proposal, not an implementation manifest. Its role is to define a coherent, teachable next-generation FunC surface that remains faithful to TVM/TOS semantics.

Introduction

Current FunC is compact and close to TVM, but that same closeness makes it hard to learn and easy to misuse. Several recurring pain points follow directly from the 0.4.6 language shape:

- domain values are not modeled as strongly as they should be;
- boolean logic, parsing success, and protocol errors are too often represented as unstructured integers;
- mutation is hidden inside special syntax such as `~method`;
- exceptions and implicit runtime conventions are used where typed return values would be easier to reason about;
- code reuse relies too heavily on low-level built-ins and ad hoc include patterns.

FunC 0.5.0 addresses these issues by keeping the low-level power of TVM available while moving ordinary authoring toward a safer and more legible surface. The intended result is not “higher level at any cost”, but a better alignment between source code and the actual constraints of on-chain execution.

Contents

1	Status and Positioning	4
1.1	Status	4
1.2	Relationship to FunC 0.4.6	4
1.3	What This Proposal Is Not	4
2	First Principles	5
2.1	Execution Model Comes First	5
2.2	Human Factors Matter	6
2.3	Non-Goals	6
3	Headline Changes with Respect to 0.4.6	7
4	Design Goals	8
4.1	Primary Goals	8
4.2	Compatibility Goals	9
5	Lexical Structure	9
5.1	Whitespace and Comments	9
5.2	Identifiers	9
5.3	Literals	10
5.4	Keywords	10
5.5	Attributes	11
6	Source Structure and Modules	11
6.1	Compilation Unit	11
6.2	Modules	12
6.3	Public API Surface	12
7	Type System	12
7.1	Built-In Scalar Types	12
7.2	Built-In Domain Types	13
7.3	Vec<T>	14
7.4	EitherRefOrInline<T>	14
7.5	Typed Cell References	15
7.6	Compound Types	15
7.7	Struct Types	16
7.8	Enum Types	16

7.9	Message Declarations	17
7.10	Option and Result	18
7.11	Type Aliases	18
7.12	Generics	19
8	Traits and Implementations	19
8.1	Trait Declarations	19
8.2	Implementations	19
8.3	Why Traits Instead of Parser Magic	20
9	Bindings, Mutability, and Patterns	20
9.1	Immutable by Default	20
9.2	Assignments	21
9.3	Pattern-Oriented Control Flow	21
9.4	Destructuring	21
9.5	Mutating Methods	22
9.6	Mutable Arguments and Lightweight Safety Checks	23
10	Functions and Effect System	24
10.1	Function Syntax	24
10.2	Pure by Default	24
10.3	Effect Sets	25
10.4	Why Not Keep <code>impure</code>	25
10.5	Entrypoints	26
10.6	Entry Dispatch Matrix	26
10.7	Entry Decoding Semantics	27
10.8	External Ininsics	28
11	Expressions and Statements	28
11.1	Core Statements	28
11.2	Match Expressions	29
11.3	Flow-Sensitive Narrowing	29
11.4	The Question-Mark Operator	29
11.5	Loop Design	30
12	Messages, Serialization, and TVM-Specific APIs	31
12.1	Typed Message Contexts	31
12.2	Typed Cell APIs	32

12.3	Codec Traits	32
12.4	Slice and Builder APIs	33
12.5	Sending Messages	34
13	Error Model	35
13.1	Recoverable and Unrecoverable Failure	35
13.2	Domain Error Types	35
13.3	Bridging Legacy Exit Codes	35
14	Standard Library Direction	36
14.1	Small Core, Rich Stdlib	36
14.2	Traits as the Unit of Reuse	36
15	What Remains Explicit	37
16	Compatibility and Migration	37
16.1	Edition Model	37
16.2	Compatibility Surface	38
16.3	Illustrative Source Migration	38
16.4	Migration Principle	38
17	Worked Comparison: One Wallet Contract in 0.4.6 and 0.5.0	39
17.1	FunC 0.4.6 Style	39
17.2	FunC 0.5.0 Style	40
17.3	What the Comparison Shows	42
17.4	Why This Example Matters	43
18	Core Grammar Summary	43
19	Illustrative Contract Fragment	45
20	Conclusion	47
21	Canonical Decisions	47
21.1	Canonical Decision 1: Stack Representation of Sum Types . . .	48
21.2	Canonical Decision 2: Wire Format of Sum Types	48
21.3	Canonical Decision 3: Layout of <code>Bytes</code>	49
21.4	Canonical Decision 4: <code>BodyLayout::Auto</code> Fit Predicate	50
21.5	Canonical Decision 5: Typed <code>Err</code> to Exit Code	50

21.6 Canonical Decision 6: Getter / <code>method_id</code> Selector Rule . . .	51
21.7 Canonical Decision 7: Expression Grammar and Precedence .	52
21.8 Canonical Decision 8: Trait Coherence	54
21.9 Canonical Decision 9: Eager Decoding on <code>entry external</code> . .	55
21.10 Canonical Decision 10: Compatibility Edition Surface	55
22 Canonical Corpus	57
A Standard Library Module Map	58

1 Status and Positioning

1.1 Status

This document is a **draft design proposal**. It defines the desired surface language, type system, and standard-library direction for a future FunC 0.5.0 edition. It does not claim that the current compiler already implements these features.

1.2 Relationship to FunC 0.4.6

FunC 0.4.6 is the implementation baseline. FunC 0.5.0 is the language-design target. The design aims to preserve the following invariants:

- compilation still targets TVM/Fift-style low-level code;
- TVM cells, slices, builders, continuations, and message actions remain first-class concerns;
- asynchronous cross-contract messaging remains the fundamental execution model;
- low-level escape hatches continue to exist for system code and stdlib authors.

1.3 What This Proposal Is Not

FunC 0.5.0 is *not* intended to be:

- a clone of Solidity;
- a full Rust compiler for TVM;
- a language that hides gas, bounce, serialization, or asynchronous delivery;
- a language that requires ownership and lifetime reasoning for ordinary contract code.

2 First Principles

2.1 Execution Model Comes First

The language must reflect the real execution model of TVM/TOS rather than a familiar but misleading model imported from EVM or general-purpose systems programming.

The relevant protocol facts are:

1. each contract owns isolated state;
2. cross-contract interaction happens through asynchronous messages;
3. a message may bounce, arrive later, or never arrive;
4. fees, reserve actions, and outgoing messages are explicit protocol operations;
5. serialization into cells is fundamental rather than incidental.

These facts impose direct source-language consequences:

1. contract-visible types must distinguish addresses, messages, coins, and raw cells;
2. fallibility should be part of the type system instead of being encoded as integer conventions;
3. mutation should be visible in the source rather than hidden in naming conventions;
4. side effects should be visible in signatures rather than collapsed into a single catch-all **impure** marker;
5. low-level TVM intrinsics should mostly live in the standard library, not in the parser.

2.2 Human Factors Matter

FunC 0.5.0 is optimized for the tasks that contract authors actually perform:

- define message schemas;
- parse inbound messages;
- update persistent state;
- send typed outbound messages;
- model recoverable failures clearly;
- reuse audited components without reading raw TVM stack code.

The language should therefore bias toward:

- readable declarations;
- local reasoning;
- exhaustiveness checks;
- effect visibility;
- error paths that are explicit and grep-able.

2.3 Non-Goals

To preserve approachability, FunC 0.5.0 deliberately excludes several features often associated with Rust:

- no ownership-and-lifetime borrow checker in the Rust sense;
- no lifetimes;
- no user-defined macros;
- no trait objects as a required abstraction mechanism;
- no `async/await`;
- no hidden heap allocation model.

At the same time, the language *may* include a lightweight checker for mutable-place overlap. Rejecting obviously dangerous expressions such as “mutate the same slice twice in one expression” is compatible with the goal of remaining easy to learn; it does not require authors to reason about lifetimes or ownership graphs.

3 Headline Changes with Respect to 0.4.6

Concern	FunC 0.4.6	FunC 0.5.0 proposal
Local binding	<code>var x = ...</code> or explicit type application syntax	<code>let x = ...</code> , <code>let mut x = ...</code> , and ordinary typed bindings
Boolean values	commonly encoded as <code>int</code>	dedicated <code>bool</code> type with no implicit int-to-bool coercion
Parsing failures	integer flags or exceptions	<code>Option<T></code> and <code>Result<T, E></code>
Mutating methods	<code>~method</code> syntax	ordinary method syntax with <code>mut self</code> in the signature
Error handling	<code>throw_*</code> helpers and <code>try/catch</code>	typed <code>Result</code> plus <code>?</code> and <code>match</code> ; raw abort still exists for low-level code
Domain types	<code>int/cell/slice/builder</code> and conventions	built-in domain types such as <code>Cell<T></code> , <code>Slice</code> , <code>Builder</code> , <code>Address</code> , <code>Coins</code> , and <code>InternalContext<T></code>
Control over impurity	single <code>impure</code> qualifier	explicit effect sets such as <code>effect(read, write, send)</code>
Message schemas	manual op-code parsing and slice plumbing	<code>enum</code> , <code>struct</code> , codec traits, and typed entry-points

Serialization	manual builder/slice plumbing for every protocol type	<code>Cell<T></code> , <code>to_cell</code> , <code>from_cell</code> , <code>load_any</code> , and <code>store_any</code> as ordinary typed APIs
Mutable alias safety	mostly unchecked beyond surface syntax	explicit mutable arguments plus a lightweight overlap checker for mutable places
Reuse	include-heavy and built-in-heavy	<code>mod</code> , <code>use</code> , generics, <code>trait</code> , and <code>impl</code>
Built-ins	many parser-predefined names	small core + stdlib <code>extern "tvm"</code> declarations

4 Design Goals

4.1 Primary Goals

FunC 0.5.0 has the following goals:

1. preserve TVM/TOS fidelity;
2. reduce incidental syntax complexity;
3. make domain concepts first-class in the type system;
4. make recoverable failure ordinary and typed;
5. make mutation and effects explicit;
6. make modules, traits, and generics the primary reuse mechanism;
7. keep low-level interop available for advanced code and compiler-adjacent libraries.

4.2 Compatibility Goals

The language should permit a staged migration from existing FunC contracts. Legacy code should not need to be rewritten in one step. Instead, the compiler should support an edition or compatibility mode in which old constructs are accepted while new constructs are gradually adopted.

5 Lexical Structure

5.1 Whitespace and Comments

FunC 0.5.0 adopts a more conventional comment system:

- `//` introduces a line comment;
- `/* ... */` introduces a nested block comment.

For migration purposes, the legacy FunC comment forms `;;` and `{- ... -}` may remain accepted in compatibility mode, but they are not part of the preferred 0.5.0 surface.

5.2 Identifiers

Ordinary identifiers use a conventional snake-case-oriented shape:

`[A-Za-z_][A-Za-z0-9_]*`

Module-qualified names use `::`:

```
std::slice::SliceReader
wallet::msg::Transfer
```

Predicate names should use `is_*` or `has_*` rather than the old `name?` convention. Fallible operations should use `try_*` or return `Result`.

5.3 Literals

The preferred literal forms are:

- integers: `0`, `17`, `0xff`, `42u32`, `-1i257`;
- booleans: `true`, `false`;
- strings: `"hello"`;
- byte strings: `b"abc"`;
- hexadecimal byte strings: `hex"deadbeef"`;
- addresses: `addr"EQC..."` or `addr"-1:abcd..."`.

The old string-suffix forms from FunC 0.4.6 may remain available behind compatibility gates, but new code should prefer explicit prefixes and typed constructors.

5.4 Keywords

The core language reserves the following new keyword set:

```
mod use pub const type struct enum message trait impl fn extern
let mut static match if else while loop for in break continue
return where self Self as
effect pure unsafe
entry external internal bounce get
```

Two of these (`static`, `where`) are reserved for future use: the language grammar in Section 18 does not currently consume them. They are listed here so that user identifiers do not collide.

Compatibility mode may additionally reserve legacy FunC words such as `global`, `asm`, `impure`, `inline`, and `method_id`.

5.5 Attributes

FunC 0.5.0 introduces an attribute syntax for edition selection and module-level metadata. Inner attributes apply to the enclosing item (here, the source file):

```

InnerAttribute ::= "#" "!" "[" AttrBody "]"
OuterAttribute ::= "#"          "[" AttrBody "]"
AttrBody      ::= identifier [ "=" (string-literal | integer-literal) ]
                  | identifier "(" [ AttrArg { "," AttrArg } ] ")"
AttrArg       ::= identifier [ "=" (string-literal | integer-literal) ]

```

The only recognised inner attribute in the edition-0.5.0 surface is `edition`:

```

#![edition = "0.5.0"]
#![edition = "0.4-compatible"]

```

Unknown attributes are reserved and reject at parse time to leave room for future extensions. Outer attributes (`#[...]`) on individual items are reserved but unused in this edition.

6 Source Structure and Modules

6.1 Compilation Unit

A source file is a sequence of items:

```

Item ::= ModuleDecl
      | UseDecl
      | ConstDecl
      | TypeAlias
      | StructDecl
      | EnumDecl
      | MessageDecl
      | TraitDecl
      | ImplDecl
      | FunctionDecl

```

6.2 Modules

Modules organize contract code and the standard library:

```
mod wallet::state;  
mod wallet::msg;  
use std::result::Result;  
use std::message::{InternalContext, SendOptions};
```

The language keeps modules simple:

- there is no macro expansion model tied to modules;
- visibility is controlled by `pub`;
- module-qualified names use `::`;
- `use` imports names into local scope.

6.3 Public API Surface

Every item is private to its module unless marked `pub`. This matters for standard library design, contract libraries, and test-only helpers:

```
pub struct WalletState { ... }  
pub enum WalletMsg { ... }  
pub message Transfer(op = 0x0f8a7ea5) { ... }  
pub fn recv_internal(...) -> Result<(), WalletError> effect(write, send) { ... }
```

7 Type System

7.1 Built-In Scalar Types

The proposed scalar types are:

```
bool  
i32  i64  i128  i256  i257  
u8   u16  u32   u64   u128  u256
```

`i257` remains available because TVM integers are fundamentally 257-bit signed values. For migration, `int` may remain an alias of `i257` in compatibility mode, but new code should prefer explicit widths.

7.2 Built-In Domain Types

The following types become first-class built-ins:

`Cell<T>`
`Slice`
`Builder`
`Cont`
`Address`
`StdAddress`
`Coins`
`Bytes`
`RawCell`
`RawSlice`
`Vec<T>`
`EitherRefOrInline<T>`

The role of these types is:

- `Cell<T>` represents a typed cell reference carrying an expected decoded payload type;
- `Slice` and `Builder` model the TVM read/write cell domain;
- `Address` models a full on-chain address, not an arbitrary slice;
- `Coins` models currency amounts and documents intent better than raw `u128`;
- `Bytes` represents ordinary byte strings;
- `RawCell` and `RawSlice` are low-level escape-hatch types for unchecked code;
- `Vec<T>` is a sequence type with a fixed stack and wire representation, usable at getter and ABI boundaries (§7.3);
- `EitherRefOrInline<T>` is a field-level codec that chooses inline or ref storage deterministically (§7.4).

7.3 `Vec<T>`

A `Vec<T>` is a homogeneous sequence usable wherever the built-in domain types are usable, including struct fields, function arguments, function returns, and `get fn` exports.

Stack representation. A `Vec<T>` on the TVM stack is a single TVM `Tuple` whose elements are the sequence's items in order. TVM tuples natively hold up to 255 elements; a `Vec<T>` longer than 255 is represented by a chain of tail tuples (last element of the outer tuple is itself a `Vec<T>` for the remainder). The chain is deterministic: each segment holds exactly `min(255, remaining)` elements before continuing.

Wire representation. When serialised into a cell, a `Vec<T>` writes:

1. `length:uint16` — the number of items, big-endian;
2. the items in order, each by `T`'s codec;
3. if the encoded items overflow the current cell (bit budget or ref budget), the encoder spills into a *tail ref* whose referent is a cell containing the remaining items, recursively.

Overflow is determined by the same fit predicate used in §21.4. The `uint16` prefix caps the wire length at 65 535 items; `Vec` values longer than that cannot be serialised and raise `CellError::Overflow` at the codec boundary.

Getter returns. A `get fn` returning `Vec<T>` exports the stack-tuple form directly to the caller. Indexers and wallets that consume the getter see a TVM tuple and can iterate without invoking a cell decode.

7.4 `EitherRefOrInline<T>`

This type represents a field whose TL-B form is `field:(Either T ^T)`: one bit of discriminant followed by either an inline encoding of `T` or a reference to a cell carrying `T`.

Stack representation. An `EitherRefOrInline<T>` on the TVM stack is a `T` (the inline/ref distinction is a codec-level concern, not a stack concern). The compiler carries no runtime discriminant past the codec boundary.

Wire representation. The encoder writes:

1. one discriminant bit (0 = inline, 1 = ref);
2. if inline: the encoded `T` immediately after the bit;
3. if ref: a single cell reference whose referent is the encoded `T`.

The choice is deterministic and uses the same fit predicate as §21.4, specialised to the current field cursor: inline iff the encoded `T` fits in the remaining bits and refs of the current cell after consuming the discriminant bit. Two compliant compilers must produce identical bytes.

`EitherRefOrInline<T>` is distinct from `Option<Cell<T>>`: the former is mandatory (`T` is always present), the latter is (0 = absent, 1 = present).

7.5 Typed Cell References

Typed cells are central to making TVM serialization tractable without hiding it:

```
pub struct WalletState {
    seqno: u32,
    owner: Address,
    plugins: Cell<PluginDict>?,
}
```

`Cell<T>` is assignable to `RawCell` when raw interop is needed, but ordinary contract code should prefer the typed form. This improves reviewability for persistent state, lazy-loaded message refs, and nested TL-B objects.

7.6 Compound Types

Compound types are:

```

(T1, T2, ..., Tn)          // tuples
[T; N]                     // fixed-size arrays
T?                          // sugar for Option<T>
Option<T>
Result<T, E>

```

Tuples remain useful because TVM naturally works with multiple stack results, but user-defined **struct** and **enum** become the preferred way to describe meaningful protocol data.

7.7 Struct Types

Structures are declared as:

```

pub struct WalletState {
    seqno: u32,
    public_key: u256,
    plugins: Dict<PluginKey, ()>,
}

```

Struct fields are named, typed, and immutable by default. Mutation happens through a mutable binding or a method with **mut self**.

7.8 Enum Types

Enumerations are algebraic data types and are central to the proposal:

```

pub enum WalletMsg {
    Transfer {
        query_id: u64,
        amount: Coins,
        to: Address,
        response_to: Option<Address>,
    },
    InstallPlugin {
        plugin: Address,
    },
    RemovePlugin {

```

```

        plugin: Address,
    },
}

```

Enums may also carry explicit discriminants when required by wire formats:

```

pub enum Op : u32 {
    Transfer = 0x0f8a7ea5,
    Burn     = 0x595f07bc,
}

```

7.9 Message Declarations

Wire-level message bodies deserve a dedicated declaration form because they are one of the dominant concepts in TVM/TOS contracts:

```

pub message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount: Coins,
    to: Address,
    response_to: Address?,
}

```

A **message** declaration behaves like a named struct plus a derived message codec. In particular:

- the opcode binding is part of the declaration rather than an external convention;
- the declaration derives the relevant cell/TL-B codec traits;
- typed entrypoints may request `InternalContext<Transfer>` directly;
- higher-level enums may still group multiple message bodies when a contract wants one sum type for dispatch.

7.10 Option and Result

Two standard prelude types are assumed. Their variant order is fixed by the canonical tag assignment in Section 21.1:

```
pub enum Option<T> {  
    None,          // tag 0  
    Some(T),       // tag 1  
}
```

```
pub enum Result<T, E> {  
    Ok(T),          // tag 0  
    Err(E),         // tag 1  
}
```

Wire-format and stack-layout are fixed canonically: `Option<T>` is bit-compatible with TL-B `Maybe X`; `Result<T, E>` uses a 1-bit tag. See Sections 21.1 and 21.2.

These types are not library afterthoughts. They are core to ordinary contract authoring:

- `Option<T>` expresses absence without sentinels;
- `Result<T, E>` expresses recoverable failure without bare exceptions;
- `match` and `?` make both types ergonomic in routine code.

For readability, `T?` is accepted as surface sugar for `Option<T>`:

```
Address?  
Cell<JettonWalletState>?
```

Unlike languages that normalize everything around `null`, the teaching path still centers `Some/None`, `match`, and `if let`.

7.11 Type Aliases

Type aliases remain important for readability:

```
pub type QueryId = u64;  
pub type PluginKey = (i32, u256);  
pub type Ton = Coins;
```

7.12 Generics

Generic functions, structs, enums, and traits use angle-bracket syntax:

```
pub struct Dict<K, V> { ... }

pub fn first<T>(items: (T, T)) -> T { ... }
```

Unlike Rust, the design does not introduce lifetime parameters or ownership bounds. Generic programming is deliberately restricted to type parameters and trait constraints.

8 Traits and Implementations

8.1 Trait Declarations

Traits provide interface-oriented reuse:

```
pub trait TlbCodec {
    fn store(self, mut b: Builder) -> Result<Builder, CellError> pure;
    fn load(mut s: Slice) -> Result<(Self, Slice), ParseError> pure;
}
```

Inside a `trait` or `impl` body, `Self` refers to the concrete type that implements (or is being defined by) the surrounding block. Outside these blocks `Self` is not in scope. A trait method whose signature does not name `self` is a **static associated function** and is called with explicit type syntax, e.g. `WalletMsg::load(cs)`. Methods with a `self` or `mut self` receiver are dispatched through ordinary method-call syntax `x.method(...)`. The proposal does *not* include borrowed receivers; there are no `&self` or `&mut self` forms.

8.2 Implementations

Implementations attach methods to types:

```
impl WalletState {
    pub fn increment_seqno(mut self) -> Self pure {
        self.seqno += 1;
    }
}
```

```

        self
    }
}

impl TlbCodec for WalletMsg {
    fn store(self, mut b: Builder) -> Result<Builder, CellError> pure { ... }
    fn load(mut s: Slice) -> Result<(Self, Slice), ParseError> pure { ... }
}

```

8.3 Why Traits Instead of Parser Magic

One design goal of 0.5.0 is to move functionality out of the parser and into audited, inspectable library code. Traits support that directly:

- codecs can be trait-based;
- message sending can depend on a trait bound rather than raw cells;
- collection and builder helpers can live in the standard library;
- test doubles and mock implementations become natural.

This does *not* mean the compiler should be blind to protocol structure. A small amount of protocol-aware lowering remains justified for things such as entripoint contexts, message opcodes, and automatic inline-vs-ref message-body placement. The goal is to eliminate accidental parser magic, not to forbid every useful built-in lowering.

9 Bindings, Mutability, and Patterns

9.1 Immutable by Default

All local bindings are immutable unless declared with `mut`:

```

let seqno: u32 = state.seqno;
let mut cs: Slice = body;

```

This replaces the old mixture of `var`, explicit type application, and hidden mutation through `~method` calls.

9.2 Assignments

Only mutable places may be assigned to:

```
let mut state = load_state()?;  
state.seqno = state.seqno + 1;
```

9.3 Pattern-Oriented Control Flow

Patterns are not limited to `match`. The language should also support `if let` and `while let` because optional values and typed message variants are ubiquitous in contract code:

```
if let Some(response_to) = msg.response_to {  
    send(response_to, Ack { query_id: msg.query_id }, opts)?;  
}  
  
while let Some(item) = queue.pop_front() {  
    process(item)?;  
}
```

9.4 Destructuring

Patterns work in `let` bindings and in `match`:

```
let (amount, response_to) = (msg.amount, msg.response_to);  
let WalletMsg::Transfer { amount, to, .. } = msg;
```

Pattern forms include:

- wildcard `_`;
- name bindings;
- tuple patterns;
- struct patterns;
- enum variant patterns.

9.5 Mutating Methods

Methods may declare `mut self`:

```
impl Slice {  
    pub fn load_uint(mut self, bits: u16) -> Result<u256, ParseError> pure { ... }  
}
```

At the call site, this looks ordinary:

```
let mut cs = body;  
let op = cs.load_uint(32)?;
```

This replaces the special `cs~load_uint(32)` syntax. The source clearly states mutability in the binding and in the method signature, rather than hiding it in punctuation.

Error-path state of mutable receivers. When a `mut self` method returns `Result<_, E>`, the canonical semantics are:

- on `Ok`, the caller-side mutable place is rebound to the updated receiver, and the `Ok` payload is propagated;
- on `Err`, the caller-side mutable place is *restored to its pre-call state*. Any partial cursor advancement or field mutation inside the callee is discarded.

This means `cs.load_uint(32)?` followed by code that reuses `cs` in the `Err` branch can rely on `cs` still pointing at the original position. The lowering in `doc/func_v0.5.0-lowering.tex` §4.3 must be read with this rule: the `unconditional writeback` shown there is a conceptual desugar step, and the emitted code must guard the writeback with a tag check, or must use a shadow copy, so that the `Err` case does not commit intermediate state. Authors may explicitly commit partial mutations by writing `let new_cs = cs; new_cs.load_uint(32)` in a non-receiver form.

9.6 Mutable Arguments and Lightweight Safety Checks

Methods with `mut self` remain the idiomatic surface for builder/slice-like APIs. Free functions — and in particular helper functions that update a contract’s persistent state — accept mutable places explicitly through a `mut` parameter qualifier:

```
pub fn load_var_uint(mut s: Slice, bits: u16) -> Result<u256, ParseError> pure {  
  
  let mut cs = body;  
  let n = load_var_uint(mut cs, 5)?;
```

Helper functions that mutate persistent state. This is the canonical pattern for splitting a contract’s entry body into smaller helpers that each mutate the protocol state. There is no `&self` or `&mut self` borrow form in FunC 0.5.0; state passes through `mut` arguments by value-thread:

```
fn on_transfer(  
  mut state: WalletState,  
  sender: Address,  
  req: Transfer,  
) -> Result<(), WalletError> effect(read, send) { ... }  
  
fn recv_internal(ctx: InternalContext<WalletMsg>) -> Result<(), WalletError> ... {  
  let mut state = load_state()?;  
  match ctx.body {  
    WalletMsg::Transfer(req) => on_transfer(mut state, ctx.sender, req)?,  
    ...  
  }  
  save_state(state)  
}
```

The lowering is identical to a method with `mut self`: the helper takes the receiver by value and returns it updated on `Ok`, rolled back on `Err` (per the rule in §21.1 and `doc/func_v0.5.0-lowering.tex` §4.3).

This pattern resolves what earlier drafts listed as “helper functions need `&mut self`”; no such syntax is required.

Overlap checks. This proposal adopts three lightweight rules:

- only mutable lvalues may be passed as `mut` arguments;
- the same mutable place may not be passed twice in one full expression;
- parent and child places may not be mutated simultaneously in one full expression.

These rules are intentionally narrow. They prevent obvious aliasing hazards without introducing ownership, region inference, or lifetime syntax.

10 Functions and Effect System

10.1 Function Syntax

The general form is:

```
[pub] fn name(arg1: T1, mut arg2: T2, ...) -> R [effect(...)] Block
```

Examples:

```
pub fn min(x: i257, y: i257) -> i257 pure { ... }
pub fn load_state() -> Result<WalletState, ParseError> effect(read) { ... }
pub fn load_uint(mut s: Slice, bits: u16) -> Result<u256, ParseError> pure { ... }
pub fn send_transfer(...) -> Result<(), SendError> effect(send) { ... }
```

10.2 Pure by Default

A function with no effect annotation is **pure**. It may compute, match, and transform values, but it may not:

- read persistent state or chain context;
- modify persistent state;
- emit outgoing messages;
- use raw unsafe TVM intrinsics.

10.3 Effect Sets

The primary effect vocabulary is:

```
effect(read)
effect(write)
effect(send)
```

These effects mean:

- **read**: may inspect persistent state, inbound context, config, time, balance, or other chain environment values;
- **write**: may persist updated state or otherwise mutate contract-owned storage;
- **send**: may emit action-phase effects such as outbound messages, reserve operations, or code changes.

An effect set is cumulative: `effect(write, send)` may both write state and send messages.

`unsafe` is *not* a member of the effect set. It is a separate top-level qualifier on a function that grants permission to use raw `extern "tvm"` intrinsics and other surface-unsafe operations. See the grammar in Section 18 for the precise distinction: a function signature may carry exactly one `EffectQualifier`, chosen from `pure`, `unsafe`, or `effect(...)`.

Effect propagation is transitive and mandatory: if a function calls another function whose effect set is E , the caller's effect set must be a superset of E (or the caller itself must be `unsafe`). A `pure` function has $E = \emptyset$ and may only call other `pure` functions. `mut self` and `mut` arguments are *not* effects; they express local state threading and are compatible with `pure`.

10.4 Why Not Keep impure

The old `impure` marker is too coarse. It does not distinguish a getter that reads block context from a function that mutates storage and sends three messages. In 0.5.0, effect sets become part of ordinary interface review.

10.5 Entrypoints

Contract-facing entrypoints are explicit:

```
entry external fn recv_external(  
    ctx: ExternalContext<SignedRequest>,  
) -> Result<(), WalletError> effect(read, write, send) { ... }  
  
entry internal fn recv_internal(  
    ctx: InternalContext<WalletMsg>,  
) -> Result<(), WalletError> effect(read, write, send) { ... }  
  
entry bounce fn on_bounce(  
    ctx: BounceContext<RawSlice>,  
) -> Result<(), WalletError> effect(read, write) { ... }
```

Getter functions use a dedicated modifier:

```
get fn seqno() -> Result<u32, ParseError> effect(read) { ... }
```

The compiler maps these to the existing TVM/TOS conventions for internal, external, bounced, and get-method execution.

The context types are not ordinary heap objects. They are compiler-recognized views over real VM inputs. Source code should nevertheless treat them as normal, typed fields:

```
ctx.sender  
ctx.value  
ctx.forward_fee  
ctx.body
```

This makes the execution model explicit while keeping raw TVM stack plumbing out of routine contract code.

10.6 Entry Dispatch Matrix

Unlike FunC 0.4.6 which uses a single `recv_internal` and expects the contract to branch on the `bounced` header bit, 0.5.0 splits internal-message delivery into two entry families:

- fresh internal messages are delivered to `entry internal`;
- bounced messages are delivered to `entry bounce`;
- external messages are delivered to `entry external`.

If a contract defines `entry internal` but no `entry bounce`, an inbound bounced message is silently dropped after consuming only the minimal gas needed by the low-level wrapper — no user-written handler is invoked. Symmetric rules apply to undefined `entry internal` (fresh internal silently dropped) and undefined `entry external` (external message rejected at the wrapper). Contracts requiring the old single-entry pattern must use a `BounceContext<RawSlice>` form and manually re-dispatch.

10.7 Entry Decoding Semantics

Typed body decoding happens in the generated wrapper, not in user code. The canonical semantics for decode failure differ per entry kind:

- `entry internal`: if the inbound body cannot be decoded as the requested `T`, the wrapper exits successfully without invoking the user body. No state is written, no messages are sent. This preserves the TOS convention that unrecognized ops do not cause bounce-refund gas drain.
- `entry external`: decoding is lazy. The wrapper exposes `ctx.body_slice: Slice` and a user-invoked `ctx.parse_body<T>() -> Result<T, ParseError>`. The wrapper does *not* decode eagerly, because the external-message gas limit (typically 10 000 gas before `accept_message`) may be exhausted by decoding a large body. Authors must call `accept_message` first, then `parse`.
- `entry bounce`: decoding uses the bounced-message body convention (at most 256 bits after the `0xFFFFFFFF` header). The default `BounceContext<RawSlice>` form accepts any bounce; typed variants must opt in and must be robust to truncation.

Bounded body width rule. For any entry `bounce fn ... (ctx: BounceContext<T>)` with `T` not equal to `RawSlice`, the compiler must statically verify

$$\text{max_encoded_bits}(T) \leq 256$$

where `max_encoded_bits` is the worst-case bit width of `T`'s derived codec, not counting refs. The `0xFFFFFFFF` bounce header prefix occupies 32 bits and is added by the wrapper, not counted against the 256-bit budget available to `T`. Ref counts are also verified: `T` must encode as purely inline bits (no refs) because bounded bodies are truncated, not referenced.

If `max_encoded_bits(T) > 256` or `T` requires any refs, the compile error is `BounceBodyTooWide`, and the author must either narrow `T`'s schema or fall back to `BounceContext<RawSlice>` and parse manually.

If a contract wants eager decoding in entry `external`, it must write entry `external(eager) fn ....` See Section 21.9 for the canonical rule.

10.8 External Ininsics

Raw TVM primitives should be declared as `extern "tvm"` in the standard library rather than being magic parser names:

```
extern "tvm" fn send_raw_message(msg: RawCell, mode: u8) -> () effect(send, unsafe)
extern "tvm" fn accept_message() -> () effect(send, unsafe);
```

This preserves power while making the surface language smaller and more teachable.

11 Expressions and Statements

11.1 Core Statements

The language supports:

```
let
let mut
if / else
if let
match
```

```
while
while let
loop
for
break
continue
return
```

11.2 Match Expressions

Pattern matching is exhaustive by default:

```
match msg {
  WalletMsg::Transfer { query_id, amount, to, response_to } => { ... }
  WalletMsg::InstallPlugin { plugin } => { ... }
  WalletMsg::RemovePlugin { plugin } => { ... }
}
```

This is a major ergonomics improvement over manually parsing an `op` code into an `int` and then maintaining a parallel maze of `if` branches.

11.3 Flow-Sensitive Narrowing

Pattern matching and optional-value tests should refine types in a flow-sensitive way:

```
if let Some(reply_to) = msg.response_to {
  send(reply_to, Receipt { query_id: msg.query_id }, opts)?;
}
```

Inside the `if let` body, `reply_to` is an `Address` rather than `Address?`. Similar narrowing should happen for `match` arms and other obvious success paths over `Option`, `Result`, and message variants.

11.4 The Question-Mark Operator

If a function returns `Result<T, E>`, the `?` operator may be applied to intermediate `Result` values:


```
let mut cs = body;
let global_id = cs.load_int(32)?;
let seqno = cs.load_uint(32)?;
let msg = WalletMsg::load(cs)?;
```

This drastically reduces the need for hand-written `throw_unless`, `try/catch`, and integer-status plumbing in ordinary code.

11.5 Loop Design

The preferred loop forms are:

```
while cond { ... }
loop { ... }
for item in items { ... }
```

Legacy `repeat` and `do ... until` may remain in compatibility mode, but the core language should converge on a smaller, more conventional control surface.

Iterator protocol for `for`. `for item in items` desugars to a `while let Some(item) = iter.next()` loop where `iter` is derived from `items` by the `Iter` trait:

```
pub trait Iter {
    type Item;
    fn next(mut self) -> Option<Self::Item> pure;
}

pub trait IntoIter {
    type Item;
    type IntoIterTy: Iter<Item = Self::Item>;
    fn into_iter(self) -> Self::IntoIterTy pure;
}
```

The desugaring rule is:

```
for <pattern> in <expr> { <body> }

==>
```

```
let mut __it = <expr>.into_iter();
while let Some(<pattern>) = __it.next() { <body> }
```

In edition 0.5.0 the compiler must accept `for` only over iterables whose type implements `IntoIter`. The standard library provides initial implementations for ranges, fixed arrays, tuples of uniform element type, and dictionary iteration; additional implementations may be added by `stdlib` or user code. Trait resolution is static (monomorphized), matching the lowering rule in `doc/func_v0.5.0-lowering.tex` §4.10.

12 Messages, Serialization, and TVM-Specific APIs

12.1 Typed Message Contexts

Messages should be represented as typed values rather than raw slices whenever possible. The standard library is expected to define context types such as:

```
pub struct InternalContext<T> {
    sender: Address,
    value: Coins,
    forward_fee: Coins,
    body: T,
}

pub struct ExternalContext<T> {
    body: T,
    signature: Option<Bytes>,
}

pub struct BounceContext<T = RawSlice> {
    sender: Address,
    value: Coins,
    original_forward_fee: Coins,
    body: T,
}
```

When an endpoint requests `InternalContext<T>` for a concrete message type, the compiler or stdlib-generated wrapper is responsible for parsing the inbound body into `T`. Contracts may still opt into `RawSlice` when they need manual parsing.

12.2 Typed Cell APIs

Serialization should be both trait-driven and convenient in routine code. Per Section 21.8, the `to_cell` convenience lives on the `CellEncode` trait itself as a default method, not as a blanket inherent impl:

```
pub trait CellEncode {
    fn encode(self) -> Result<RawCell, CellError> pure;
    fn to_cell(self) -> Result<Cell<Self>, CellError> pure {
        self.encode().map(Cell::from_raw)
    }
}

impl<T: CellDecode> Cell<T> {
    pub fn load(self) -> Result<T, ParseError> pure;
    pub fn begin_parse(self) -> Slice pure;
}
```

The `impl<T: CellDecode> Cell<T> { ... }` form *is* admissible because `Cell` is declared in the standard library (the trait's originating crate). User code may not add its own blanket impls over foreign types.

12.3 Codec Traits

Serialization and deserialization should be expressed through traits:

```
pub trait CellEncode {
    fn encode(self) -> Result<RawCell, CellError> pure;
}

pub trait CellDecode: Sized {
    fn decode(cell: RawCell) -> Result<Self, ParseError> pure;
}
```

For wire protocols that rely on TL-B, a narrower `TlbCodec` trait may specialize the same idea.

The standard library should also offer high-frequency helpers modeled directly on how contracts are written:

```
impl Slice {
  pub fn load_any<T: CellDecode>(mut self) -> Result<T, ParseError> pure;
}

impl Builder {
  pub fn store_any<T: CellEncode>(mut self, value: T) -> Result<Self, CellError>
}
```

12.4 Slice and Builder APIs

The low-level APIs remain essential, but their contracts become explicit:

```
impl Slice {
  pub fn load_uint(mut self, bits: u16) -> Result<u256, ParseError> pure;
  pub fn load_int(mut self, bits: u16) -> Result<i257, ParseError> pure;
  pub fn load_ref(mut self) -> Result<RawCell, ParseError> pure;
  pub fn load_addr(mut self) -> Result<Address, ParseError> pure;
  pub fn remaining_bits(self) -> u16 pure;
  pub fn remaining_refs(self) -> u8 pure;
}

impl Builder {
  pub fn store_uint(mut self, value: u256, bits: u16) -> Result<Self, CellError>
  pub fn store_int(mut self, value: i257, bits: u16) -> Result<Self, CellError>
  pub fn store_ref(mut self, cell: RawCell) -> Result<Self, CellError> pure;
  pub fn end_cell(self) -> Result<RawCell, CellError> pure;
}
```

Note that parsing and building remain explicit operations. The language does not try to make TVM cells disappear; it makes them typed and tractable.

12.5 Sending Messages

High-level message sending should take typed bodies:

```
pub enum BounceMode {
    NoBounce,
    Body256,
    Rich,
}

pub enum BodyLayout {
    Auto,
    Inline,
    ByRef,
}

pub struct SendOptions {
    value: Coins,
    bounce: BounceMode,
    body_layout: BodyLayout,
    mode: SendMode,
}

pub fn send<T: CellEncode>(
    dst: Address,
    body: T,
    opts: SendOptions,
) -> Result<(), SendError> effect(send);
```

This API expresses a real protocol operation more directly than ad hoc raw-cell construction plus `SENDRAWMSG`.

When `body_layout` is `Auto`, the compiler *must* choose between inline and ref according to the canonical fit predicate specified in Section 21.4. The predicate is a deterministic function of `(env_bits, env_refs, body_bits, body_refs)` and is stable across toolchains: two compliant compilers compiling the same source produce identical wire bytes. Using “may” here instead of “must” would silently break message hashing, fee estimation, and receiver-side parsing across contracts built with different toolchains.

When `body` is the unit value `()`, the generated envelope encodes “no body”: the TL-B `Maybe` bit for `body: (Maybe \ \^{}Cell)` is set to 0. This is distinct from a zero-width inlined body or an empty cell reference.

13 Error Model

13.1 Recoverable and Unrecoverable Failure

FunC 0.5.0 distinguishes:

- recoverable failures, represented by `Result<T, E>`;
- unrecoverable VM aborts, represented by `abort` or raw unsafe intrinsics.

The default teaching path should strongly prefer typed recoverable failure.

13.2 Domain Error Types

Contracts are expected to define domain-specific error enums:

```
pub enum WalletError {
    InvalidSignature,
    WrongGlobalId,
    WrongSeqno,
    UnknownPlugin,
    Parse(ParseError),
    Send(SendError),
}
```

13.3 Bridging Legacy Exit Codes

Compatibility layers may map legacy numeric exit codes to domain enums:

```
pub enum LegacyExitCode : u16 {
    InvalidSignature = 202,
    WrongSeqno = 33,
}
```

This lets modern source code remain typed even when it interoperates with legacy tooling or protocol conventions.

14 Standard Library Direction

14.1 Small Core, Rich Stdlib

The language core should stay small. Most functionality that is currently special or parser-defined should instead move to the standard library:

- arithmetic helper functions;
- slice and builder utilities;
- dictionary abstractions;
- message and context types;
- codec traits and implementations;
- contract-state helpers;
- cryptographic intrinsics surfaced as `extern "tvm"` functions.

Expected stdlib conveniences include:

- `create_message` and `send` wrappers over raw action lists;
- typed state load/save helpers built on `Cell<T>`;
- `load_any` / `store_any` helpers for routine TL-B work;
- message-body declarations with derived codecs.

14.2 Traits as the Unit of Reuse

Traits provide the right level of abstraction for:

- codecs;
- formatting and debugging;
- message sending;
- dictionary key/value constraints;

- reusable protocol behaviors.

This gives TOS a realistic path toward an audited contract standard library without forcing all composition through includes and naming conventions.

15 What Remains Explicit

The proposal intentionally keeps several low-level concerns visible:

- serialization into cells;
- bounce handling;
- coins attached to outgoing messages;
- effectful operations such as `send` and reserve actions;
- unsafe raw TVM interop where needed.

This is a language for TVM/TOS, not an attempt to hide the platform.

16 Compatibility and Migration

16.1 Edition Model

The recommended migration path is edition-based. A source file may opt into the new syntax and semantics explicitly:

```
#![edition = "0.5.0"]
```

Legacy files may continue to compile under a compatibility edition:

```
#![edition = "0.4-compatible"]
```


16.2 Compatibility Surface

The compatibility edition may continue to accept:

- legacy type names such as `int` and `cell`;
- legacy comments;
- legacy `global`, `const`, `asm`, `impure`, `inline`, `method_id` forms;
- legacy `~method` mutation syntax;
- legacy exception-style helpers.

16.3 Illustrative Source Migration

```
// 0.4.6 style
var cs = in_msg;
var signature = in_msg~load_bits(512);
throw_unless(33, msg_seqno == stored_seqno);
```

```
// 0.5.0 style
let mut cs = in_msg;
let signature = cs.load_bits(512)?;
if msg_seqno != stored_seqno {
    return Err(WalletError::WrongSeqno);
}
```

16.4 Migration Principle

The important migration principle is semantic, not cosmetic:

- move from raw integers toward explicit types;
- move from hidden mutation toward `mut`;
- move from integer-status parsing toward `Result`;
- move from raw op-code dispatch toward `enum + match`;
- move from parser magic toward `stdlib` declarations, while retaining a small amount of protocol-aware lowering where it clearly improves source-level clarity.

17 Worked Comparison: One Wallet Contract in 0.4.6 and 0.5.0

The following comparison shows the same piece of contract logic in both language styles:

1. load wallet state;
2. parse an inbound transfer request;
3. validate the transfer sequence number;
4. send native coins to the destination;
5. increment and persist the sequence number.

The examples are intentionally compact. They are not meant to be full production wallets; they are meant to expose the source-language shape of the same task.

17.1 FunC 0.4.6 Style

In current-style FunC, the code is dominated by raw slice parsing, implicit mutation through `~method` syntax, integer-coded failures, and manual message construction:

`;; FunC 0.4.6 style`

```
(int, slice) load_data() inline {
  var ds = get_data().begin_parse();
  return (ds~load_uint(32), ds~load_msg_addr());
}
```

```
() save_data(int seqno, slice owner) impure inline {
  set_data(
    begin_cell()
      .store_uint(seqno, 32)
      .store_slice(owner)
      .end_cell()
```

```

    );
}

() recv_internal(slice in_msg_body) impure {
  var (stored_seqno, owner) = load_data();
  var cs = in_msg_body;

  int op = cs~load_uint(32);
  throw_unless(40, op == 0x0f8a7ea5);

  int msg_seqno = cs~load_uint(32);
  slice to = cs~load_msg_addr();
  int amount = cs~load_tomis();

  throw_unless(33, msg_seqno == stored_seqno);
  throw_unless(34, amount > 0);

  send_raw_message(
    begin_cell()
      .store_uint(0x18, 6)
      .store_slice(to)
      .store_tomis(amount)
      .store_uint(0, 1 + 4 + 4 + 64 + 32 + 1 + 1)
      .end_cell(),
    1
  );

  save_data(stored_seqno + 1, owner);
}

```

17.2 FunC 0.5.0 Style

In the proposed 0.5.0 surface, the same logic is expressed through typed state, a typed message declaration, explicit mutability, explicit effects, and a **Result**-based error path:

```
#![edition = "0.5.0"]
```

```

pub struct WalletState {
    seqno: u32,
    owner: Address,
}

pub message Transfer(op = 0xf8a7ea5) {
    seqno: u32,
    to: Address,
    amount: Coins,
}

pub enum WalletError {
    WrongSeqno,
    ZeroAmount,
    Parse(ParseError),
    Send(SendError),
}

pub fn load_state() -> Result<WalletState, ParseError> effect(read) { ... }
pub fn save_state(state: WalletState) -> Result<(), CellError> effect(write) { ... }

entry internal fn recv_internal(
    ctx: InternalContext<Transfer>,
) -> Result<(), WalletError> effect(read, write, send) {
    let mut state = load_state()?;
    let transfer = ctx.body;

    if transfer.seqno != state.seqno {
        return Err(WalletError::WrongSeqno);
    }
    if transfer.amount.is_zero() {
        return Err(WalletError::ZeroAmount);
    }

    send(
        transfer.to,
        (),
        SendOptions {

```

```

        value: transfer.amount,
        bounce: BounceMode::Body256,
        body_layout: BodyLayout::Auto,
        mode: SendMode::PayFeesSeparately,
    }
)?;

state.seqno += 1;
save_state(state)?;
Ok(())
}

```

17.3 What the Comparison Shows

The comparison illustrates the intended source-level improvements:

- **Typed message declaration instead of raw op-code dispatch.**
In 0.4.6 the contract manually loads `op`, then loads each field from a `slice`. In 0.5.0 the `Transfer` message binds its opcode in the declaration and the entrypoint receives a typed body directly.
- **Explicit domain types.** `Address` and `Coins` replace raw `slice` and `int` conventions, making the meaning of each field visible at the type level.
- **Explicit mutation.** In 0.4.6 mutation is hidden inside `cs~load_uint(...)` and similar forms. In 0.5.0 mutation is carried by `let mut cs` or `let mut state` and by explicit mutable parameters.
- **Typed recoverable errors instead of numeric abort codes.** `throw_unless(33, ...)` and `throw_unless(34, ...)` become `Err(WalletError::WrongSeqno)` and `Err(WalletError::ZeroAmount)`. This makes reviews and refactors much easier.
- **High-level send API instead of manual action-cell assembly.**
The 0.4.6 version has to build an outbound message cell explicitly. The 0.5.0 version exposes the protocol operation as `send(...)`, with `value`, `bounce mode`, `body layout`, and `mode` still explicit.

- **Persistent state becomes a typed struct.** The state layout is no longer “a sequence of manually loaded fields” from the reader’s point of view. The source speaks in terms of `WalletState`.

17.4 Why This Example Matters

This example is deliberately small, but the same pattern scales to larger contracts:

- Jetton and NFT messages become enums instead of raw op-code trees;
- single message bodies can be declared with bound opcodes instead of parallel “struct + opcode constant” conventions;
- parser helpers return `Result` instead of integer flags;
- bounce handlers match typed variants instead of re-parsing raw slices;
- audited stdlib modules provide `send`, codecs, and dict abstractions instead of every contract open-coding them.

18 Core Grammar Summary

The grammar below is intentionally schematic. It describes the shape of the language, not every precedence rule or parsing ambiguity.

```
SourceFile ::= { InnerAttribute } { Item }
```

```
Item ::= { OuterAttribute }
      ( ModuleDecl
      | UseDecl
      | ConstDecl
      | TypeAlias
      | StructDecl
      | EnumDecl
      | MessageDecl
      | TraitDecl
      | ImplDecl
      | FunctionDecl )
```

```

ModuleDecl ::= "mod" Path ";"
UseDecl    ::= "use" Path [ "as" identifier ] ";"
ConstDecl  ::= [ "pub" ] "const" identifier ":" Type "=" Expr ";"
TypeAlias  ::= [ "pub" ] "type" identifier [ GenericParams ] "=" Type ";"

StructDecl ::= [ "pub" ] "struct" identifier [ GenericParams ] "{ Fields }"
EnumDecl   ::= [ "pub" ] "enum" identifier [ GenericParams ] [ ":" Type ] "{ Variants }"
MessageDecl ::= [ "pub" ] "message" identifier [ "(" "op" "=" Expr ")" ] "{ Fields }"
TraitDecl  ::= [ "pub" ] "trait" identifier [ GenericParams ] "{ TraitItems }"
ImplDecl   ::= "impl" [ GenericParams ] Type [ "for" Type ] "{ ImplItems }"

FunctionDecl ::= [ "pub" ] [ EntryQualifier ] "fn" identifier
               [ GenericParams ] "(" Params ")" "->" Type
               [ EffectQualifier ] BlockOrSemicolon

EntryQualifier ::= "entry" ( "external" | "internal" | "bounce" )
                  | "get"

EffectQualifier ::= "pure"
                  | "unsafe"
                  | "effect" "(" EffectList ")"

Stmt ::= LetStmt
       | ExprStmt
       | ReturnStmt
       | IfStmt
       | IfLetStmt
       | MatchStmt
       | WhileStmt
       | WhileLetStmt
       | LoopStmt
       | ForStmt
       | BreakStmt

```

```

    | ContinueStmt
    | Block

Type ::= SimpleType
      | Type "?"

Arg ::= Expr
      | "mut" Expr

```

19 Illustrative Contract Fragment

The following fragment demonstrates the overall style target:

```

#![edition = "0.5.0"]

use std::result::Result;
use std::message::{InternalContext, SendOptions, SendMode, send};

pub struct WalletState {
    seqno: u32,
    owner: Address,
}

pub enum WalletError {
    WrongSeqno,
    Parse(ParseError),
    Send(SendError),
}

pub message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount: Coins,
    to: Address,
    response_to: Address?,
}

pub fn load_state() -> Result<WalletState, ParseError> effect(read) { ... }

```



```

pub fn save_state(state: WalletState) -> Result<(), CellError> effect(write) { ...

entry internal fn recv_internal(
    ctx: InternalContext<Transfer>,
) -> Result<(), WalletError> effect(read, write, send) {
    let mut state = load_state()?;
    let msg = ctx.body;

    if let Some(response_to) = msg.response_to {
        send(
            response_to,
            Receipt { query_id: msg.query_id },
            SendOptions {
                value: 0.into(),
                bounce: BounceMode::NoBounce,
                body_layout: BodyLayout::Auto,
                mode: SendMode::PayFeesSeparately,
            },
        )?;
    }

    send(
        msg.to,
        msg,
        SendOptions {
            value: msg.amount,
            bounce: BounceMode::Body256,
            body_layout: BodyLayout::Auto,
            mode: SendMode::PayFeesSeparately,
        },
    )?;

    state.seqno = state.seqno + 1;
    save_state(state)?;
    Ok(())
}

```

20 Conclusion

FunC 0.5.0 should become easier to read not by hiding TVM/TOS, but by modeling it honestly:

- typed messages instead of raw slices;
- enums and exhaustive `match` instead of integer dispatch;
- message declarations with bound opcodes instead of ad hoc constants;
- typed cell references and auto codec helpers instead of repetitive builder/slice plumbing;
- `Result` and `Option` instead of status flags and surprise exceptions;
- `let mut` and `mut self` instead of punctuation-based mutation;
- `if let` and flow-sensitive narrowing for common optional/message control flow;
- explicit effects instead of a single opaque impurity bit;
- trait- and module-based reuse instead of parser magic and ad hoc includes.

That combination preserves the strengths of FunC — closeness to TVM, direct control over serialization and messaging, and efficient compilation — while making the source language much easier to teach, review, and scale across a real contract ecosystem.

21 Canonical Decisions

The items below were open questions in an earlier draft. Each has now been fixed from first principles: minimum runtime cost, determinism across toolchains, fidelity to TVM/TOS, and alignment with existing TON conventions. Compilers claiming specification conformance must obey these rules exactly.

21.1 Canonical Decision 1: Stack Representation of Sum Types

An `enum`, `Option<T>`, or `Result<T, E>` value occupies exactly two TVM stack values:

`(tag : int, payload : tuple)`

- `tag` is an ordinary TVM 257-bit signed integer. No bit packing is performed on the stack; that is a wire-format concern only.
- `payload` is an ordinary TVM tuple. Variant fields appear in declaration order. A zero-field variant uses the empty tuple `[]`. A one-field variant uses the singleton tuple `[v]`. Multi-field variants use tuples of the appropriate arity.

Tag values:

- `Option<T>`: `None = 0`, `Some = 1`.
- `Result<T, E>`: `Ok = 0`, `Err = 1`.
- User-defined enums without an explicit discriminant type: tags are assigned `0`, `1`, `2`, ... in declaration order.
- User-defined enums with an explicit discriminant type (`enum E : uN`) or explicit per-variant values: the declared numeric values are the stack tags.

Optimisers may locally unbox the payload when the unboxing is not observable at function, trait, or ABI boundaries, but the canonical model above is what every compiler must be able to produce on demand (for example at `store_any` boundaries).

21.2 Canonical Decision 2: Wire Format of Sum Types

The codec for a sum type S with N variants writes:

1. a tag prefix of width

$$w_S = \max(1, \lceil \log_2 N \rceil)$$

bits, big-endian, unsigned, unless the enum declares an explicit discriminant type : `uN` in which case the tag is exactly `uN` wide;

2. the variant fields, in declaration order, each by its own codec.

For `Option<T>` the tag is 1 bit (`None = 0`, `Some = 1`), yielding a wire format bit-compatible with TL-B `Maybe X`. For `Result<T, E>` the tag is 1 bit (`Ok = 0`, `Err = 1`).

A message `M(op = X)` declaration is a special case: the wire prefix is the 32-bit opcode `X`, not a derived enum tag. A sum of `message` variants (for dispatch) uses the 32-bit opcodes of each constituent message as its discriminant; the enum does not insert an additional tag.

Adding, removing, or reordering variants of an enum *breaks the wire format*. Implementations must reject silent breakage at compile time by computing a schema hash over the enum definition and including it in the generated ABI manifest.

21.3 Canonical Decision 3: Layout of Bytes

`Bytes` is a `stdlib` struct with a fixed ABI:

- **Stack:** a single TVM `Slice` that is guaranteed byte-aligned (`remaining_bits() % 8 == 0`) when entering any function that accepts `Bytes`.
- **Wire:** the “snake” cell-chain convention already standard in TON: up to $\lfloor (1023 - \text{header bits}) / 8 \rfloor$ bytes inline in the current cell, overflow spilled into a single tail reference whose referent is itself a snake cell. A length prefix is *not* part of the snake convention; the reader stops at end-of-slice.
- For protocols that need a length-prefixed byte string, `stdlib` provides `LenPrefixedBytes` with an explicit `u16` length.

`Bytes` is deliberately not a `Cell<Bytes>` because the most common pattern (message bodies, metadata strings) inlines the prefix into the current cell rather than forcing a ref.

21.4 Canonical Decision 4: BodyLayout::Auto Fit Predicate

Let `env` be the current envelope builder after the compiler has stored the common message header (source, destination, value, flags, created-lt, created-at, init, and the outer `Either` tag for the body). Let `body` be the encoded body after materialising it into a temporary builder. Define:

```
env_bits   = env.bits_used()
env_refs   = env.refs_used()
body_bits  = body.bits_used()
body_refs  = body.refs_used()
```

`BodyLayout::Auto` emits the *inline* form `(body:(Either X \^{}X) = Left X)` iff both of:

```
env_bits + 1 + body_bits <= 1023
env_refs +      body_refs <=      4
```

Otherwise it emits the *by-ref* form `(Right \^{}X)`. The constant +1 accounts for the outer `Either` tag bit. All compilers must compute exactly this predicate.

When `body_bits` is statically known (constant-size codec), the compiler may fold the branch at compile time. When it is not, the compiler emits a runtime branch whose test matches the predicate above bit-for-bit.

`BodyLayout::Inline` and `BodyLayout::ByRef` force the respective choice; a body that does not fit inline under `Inline` is a runtime error mapped to `CellError::Overflow`.

21.5 Canonical Decision 5: Typed Err to Exit Code

When the user body of an `entry` function returns `Err(e)`:

- persistent state is **not** committed (no `set_data`);
- queued action-phase effects are **not** committed (the action list built during the user body is discarded);
- the contract terminates via `THROW n` where `n` is the *exit code* derived from `e`.

The exit code derivation is:

1. If the error type is an `enum`, and each variant carries an explicit unsigned discriminant within `[1, 65535]` (directly via `enum E : uN` and `per-variant = N`), the discriminant is the exit code.
2. Otherwise the compiler assigns `exit_code(variant_i) = 1000 + i` where `i` is the zero-based declaration index. The range `[1000, 65535]` is the user range.
3. The range `[0, 999]` is reserved: `0` is success, `1` through `999` are reserved for TVM, TOS, and compiler-internal use. Explicit user discriminants in this range are a compile-time error.

Variants that carry additional payload data (e.g. `Parse(ParseError)`) use only their own variant discriminant as the exit code; the nested payload is discarded at the exit boundary. Contracts that need richer error telemetry must emit it explicitly through `send` or getters before returning `Err`.

On `Ok`, the wrapper commits state (if the user called `save_state`) and action-phase effects normally, then returns `0`.

21.6 Canonical Decision 6: Getter / method_id Selector Rule

For every `get fn name(...)`:

- the default exported method id is `crc16(name) \ 0x10000` (identical to FunC 0.4.6, preserving interop with every existing TON/TOS indexer and wallet);
- an explicit override uses the same syntax family as 0.4.6: `get fn name(...) method_id(N)` for a literal integer, or `method_id("string")` to select via the `crc16` of an alternative name;
- collisions in the final exported method-id set are a compile-time error; the compiler reports every colliding pair;
- two sibling compilations that produce different method-id assignments for the same source are a violation of this rule.

The grammar for `FunctionDecl` is extended accordingly:

```
FunctionDecl ::= [ "pub" ] [ EntryQualifier ] "fn" identifier
               [ GenericParams ] "(" Params ")" "->" Type
               [ EffectQualifier ]
               [ "method_id" ( "(" IntLiteral ")"
                             | "(" StringLiteral ")" ) ]
               BlockOrSemicolon
```

`method_id` is admissible only on `get fn` functions in edition 0.5.0; using it on a non-getter is a compile-time error.

21.7 Canonical Decision 7: Expression Grammar and Precedence

The expression grammar below supersedes the earlier schematic summary. It is parsed top-down with precedence climbing.

```
Expr      ::= Assign
Assign    ::= Or [ AssignOp Assign ]           // right-assoc
AssignOp  ::= "=" | "+=" | "-=" | "*=" | "/=" | "%="
           | "&=" | "|=" | "^=" | "<<=" | ">>="

Or        ::= And { "|" And }                 // left-assoc, short-circuit
And       ::= Compare { "&&" Compare }         // left-assoc, short-circuit
Compare   ::= BitOr [ CmpOp BitOr ]           // non-associative
CmpOp     ::= "==" | "!=" | "<" | ">" | "<=" | ">=" | "<=>"

BitOr     ::= BitXor { "|" BitXor }           // left-assoc
BitXor    ::= BitAnd { "^" BitAnd }           // left-assoc
BitAnd    ::= Shift { "&" Shift }             // left-assoc
Shift     ::= Add { ( "<<" | ">>" ) Add }      // left-assoc
Add       ::= Mul { ( "+" | "-" ) Mul }       // left-assoc
Mul       ::= Unary { ( "*" | "/" | "%" ) Unary }

Unary     ::= { UnaryOp } Cast
UnaryOp   ::= "-" | "!" | "~"
Cast      ::= Postfix [ "as" Type ]
```

```

Postfix    ::= Primary { PostfixOp }
PostfixOp  ::= "?"                                // Result/Option propagation
              | "." Ident [ "(" Args ")" ]        // field or method call
              | "(" Args ")"                      // function call
              | "[" Expr "]"                      // index
              | "::" Ident                        // associated fn/const path

Primary    ::= Literal
              | "(" ")" | "(" Expr { "," Expr } ")"
              | "[" "]" | "[" Expr { "," Expr } "]"
              | StructLit
              | EnumLit
              | Path
              | "if" Expr Block [ "else" Block ]
              | "match" Expr "{" { MatchArm } "}"
              | Block

Args       ::= [ Arg { "," Arg } ]
Arg        ::= Expr | "mut" Expr

```

Precedence summary (highest to lowest):

Level	Forms	Associativity
14	postfix: <code>?</code> , <code>.</code> , <code>(..)</code> , <code>[...]</code> , <code>::</code>	left
13	prefix: <code>-</code> , <code>!</code> , <code>~</code> ; <code>as</code>	right
12	<code>*</code> / <code>%</code>	left
11	<code>+</code> -	left
10	<code><<</code> <code>>></code>	left
9	<code>&</code>	left
8	<code>^</code>	left
7	<code> </code>	left
6	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> <code><=></code>	non-associative
5	<code>&&</code>	left (short-circuit)
4	<code> </code>	left (short-circuit)
2	<code>=</code> and compound assignments	right

Notes:

- Comparisons are non-chainable: `a < b < c` is a syntax error.
- `?` binds tighter than method-call dot: `x?.foo()` means `(x?).foo()`. Writing `x.foo()?` propagates after calling.
- `as` takes a type, not an expression, on the right.
- Bitwise `&/|/^` coexist with logical `&&/||` at separate precedences. Logical operators require `bool` operands and short-circuit; bitwise operators work on integer types only.

21.8 Canonical Decision 8: Trait Coherence

FunC 0.5.0 adopts an orphan-style coherence rule:

- `impl Trait for T` is admissible iff `Trait` *or* `T` is declared in the current compilation unit (the file or the set of files forming one `mod`).
- A blanket `impl<X: Bound> Trait for X` is admissible iff `Trait` is declared in the current compilation unit.
- For any concrete pair (`Trait`, `T`) there may be at most one `impl` in the full dependency graph; a second `impl` is a compile-time error. This includes indirect duplication through blanket impls.
- The earlier `impl<T: CellEncode> T { fn to_cell(...) }` shorthand in §7 is not admissible. `to_cell` is instead a method on the `CellEncode` trait itself, either abstract or as a default method using the trait's `encode`:

```
pub trait CellEncode {
    fn encode(self) -> Result<RawCell, CellError> pure;
    fn to_cell(self) -> Result<Cell<Self>, CellError> pure {
        self.encode().map(Cell::from_raw)
    }
}
```

Coherence is checked after monomorphisation: an instantiated concrete pair must not acquire two impls via different routes.

21.9 Canonical Decision 9: Eager Decoding on entry external

By default `entry external fn foo(ctx: ExternalContext)` receives an untyped `ctx`:

```
pub struct ExternalContext {
  body_slice: Slice,
  signature: Option<Bytes>,
}

impl ExternalContext {
  pub fn parse_body<T: CellDecode>(mut self)
    -> Result<T, ParseError> pure { ... }
}
```

Authors call `parse_body` after `accept_message`, so decoding gas is paid from contract balance rather than from the pre-accept 10 000-gas external-message quota.

Authors who accept the gas risk may opt into eager decoding:

```
entry external(eager) fn recv_external(
  ctx: ExternalContext<SignedRequest>,
) -> Result<(), WalletError> effect(read, write, send) { ... }
```

The grammar for `EntryQualifier` is extended:

```
EntryQualifier ::= "entry" "external" [ "(" "eager" ")" ]
                | "entry" "internal"
                | "entry" "bounce"
                | "get"
```

For `entry internal` and `entry bounce` the default is already eager decoding; no opt-in is required.

21.10 Canonical Decision 10: Compatibility Edition Surface

An edition is chosen per source file by an inner attribute; it is not per-item.

- `#[edition = "0.5.0"]` selects the strict 0.5.0 surface. This is the recommended edition for new code.
- `#[edition = "0.4-compatible"]` selects the superset edition that accepts all 0.4.6 constructs plus all 0.5.0 constructs. The declared purpose is migration, not long-term authoring.
- Absence of an edition attribute defaults to 0.4-compatible for this release, so existing contracts continue to compile without modification.

Under 0.4-compatible the following legacy constructs are accepted in addition to the 0.5.0 surface:

- lexical: `;` line comments; `{- ... -}` nested block comments; legacy string suffixes `s`, `a`, `u`, `h`, `H`, `c`; `?`-suffixed predicate identifiers; `~`-prefixed modifying methods;
- top-level: `#pragma ...`, `#include ...`, global declarations, `const` declarations without type, function bodies introduced by `asm` with `method_id` after modifiers;
- function modifiers: `impure`, `inline`, `inline_ref`, `method_id` in 0.4.6 positions;
- types: `int` as an alias for `i257`, and lowercase `cell`, `slice`, `builder`, `cont`, `tuple` as aliases for the 0.5.0 domain types;
- statements: `repeat Expr Block, do Block until Expr`;
- expressions: `var ... bindings`, explicit type application `(int, slice) (n, s) = ...` on the left of `=`, bare unit return `return ()`; method calls of the form `receiver~name(args)`.

Cross-edition interoperation:

- a 0.5.0 file may use items from a 0.4-compatible file, provided the imported items have types expressible in the 0.5.0 surface;
- a 0.4-compatible file may call 0.5.0 items that do not use features absent from the compat surface (notably: `Result`, typed `entry`, traits, or enums);

- a `0.4-compatible` file may *not* define a `0.5.0 entry` function; entry points cross the compatibility boundary exactly once, at the outer edition of the contract's root module;
- the contract's root module's edition determines the emitted ABI.

Removal schedule. `0.4-compatible` is a migration surface, not a long-term supported edition. It is scheduled for removal in a future edition `0.7.0`. Contracts that have not migrated by then must freeze on the last compiler release that still accepts it.

22 Canonical Corpus

The 10 Canonical Decisions above are normative on paper; the corpus is how they are validated against real contracts and how cross-compiler conformance is checked. The corpus is defined in a sibling document, `doc/func_v0.5.0-corpus.md`, which ships as version-pinned artefacts under `crypto/smartcont/v050/`.

Every compliance-seeking compiler must produce byte-identical TVM code-cell output for every corpus entry at its pinned version. Corpus v1 includes:

- C1 — Simple wallet (`wallet3`);
- C2 — Plugin wallet (`wallet-v4`);
- C3 — Highload wallet v2;
- C4 — Jetton wallet (synthetic, from TEP-74);
- C5 — Multisig (sketch; full rewrite in corpus v2);
- — Elector (deferred to corpus v2).

The corpus is intentionally small in v1: enough to pin the language, not so large that freezing it blocks edition maturation. Corpus versions are monotonic — v2 may add entries but may not redefine entries already fixed in v1, short of a spec-revision-grade compatibility break.

Issues surfaced by corpus v1, now resolved in main spec v3. Every issue that corpus v1 flagged as open has been folded into the main specification:

- helper-function state mutation — resolved in §9.6 by the existing `mut` parameter qualifier; there is no `\&mut self` syntax;
- `Vec<T>` at getter and ABI boundaries — resolved in §7.3;
- `EitherRefOrInline<T>` per-field encoding — resolved in §7.4;
- bounce-body width enforcement — resolved in §10.7 by the `BounceBodyTooWide` compile-time check;
- canonical `std::` module layout — resolved in Appendix A below.

A Standard Library Module Map

The canonical `std::` module tree below is part of the edition-0.5.0 ABI. Every conforming compiler must accept these module paths and export the listed names. Item signatures are authoritative in the modules themselves (shipped alongside the reference compiler); this appendix fixes the *tree* so that `use` paths across source bases are compatible.

Module path	Key exports
<code>std::prelude</code>	auto-imported into every source file: <code>Result</code> , <code>Option</code> , <code>Ok</code> , <code>Err</code> , <code>Some</code> , <code>None</code> ; the <code>?</code> lowering relies on these exact names
<code>std::cell</code>	<code>Cell<T></code> , <code>RawCell</code> , <code>Builder</code> , <code>Slice</code> , <code>RawSlice</code> , <code>CellError</code> , <code>ParseError</code> ; conversions <code>to_cell</code> , <code>from_cell</code> , <code>load_any</code> , <code>store_any</code> ; <code>Cell::from_raw</code> , <code>Cell::as_raw</code>
<code>std::address</code>	<code>Address</code> , <code>StdAddress</code> , <code>parse_std_addr</code> , <code>Address::derive</code> , address codecs
<code>std::coins</code>	<code>Coins</code> , <code>Coins::zero</code> , <code>Coins::is_zero</code> , arithmetic, safe conversions

Module path	Key exports
<code>std::bytes</code>	<code>Bytes</code> , <code>LenPrefixedBytes</code> , helpers for snake encoding and byte-alignment assertions
<code>std::vec</code>	<code>Vec<T></code> , <code>EitherRefOrInline<T></code> ; iteration traits on <code>Vec</code>
<code>std::dict</code>	<code>Dict<K, V></code> , <code>DictEntry<K, V></code> , iteration via <code>iter_sorted</code> , <code>Dict::pop_min</code> , <code>Dict::contains</code> , <code>Dict::insert</code> , <code>Dict::remove</code> , <code>Dict::get</code>
<code>std::crypto</code>	<code>check_signature</code> , <code>slice_hash</code> , <code>sha256</code> , <code>keccak256</code> , curve primitives that expose TVM hashing opcodes
<code>std::chain</code>	<code>now</code> , <code>global_id</code> , <code>accept_message</code> , <code>raw_reserve</code> , <code>get_data</code> , <code>set_data</code> , contract-balance and block-context readers
<code>std::message</code>	<code>send</code> , <code>send_raw</code> , <code>SendOptions</code> , <code>SendMode</code> , <code>BounceMode</code> , <code>BodyLayout</code> , <code>InternalContext<T></code> , <code>ExternalContext</code> , <code>BounceContext<T></code> , <code>SendError</code>
<code>std::iter</code>	<code>Iter</code> , <code>IntoIter</code> traits (see §21.7); stdlib iterator impls for <code>Vec</code> , <code>Dict</code> , ranges, fixed arrays, uniform tuples
<code>std::result</code>	re-exports from <code>std::prelude</code> ; additional combinators <code>map</code> , <code>map_err</code> , <code>and_then</code> , <code>or_else</code> , <code>ok_or</code> , <code>expect</code>
<code>std::option</code>	re-exports from <code>std::prelude</code> ; additional combinators <code>map</code> , <code>unwrap</code> , <code>unwrap_or</code> , <code>ok_or</code> , <code>is_some</code> , <code>is_none</code>
<code>std::intrinsics</code>	low-level <code>extern "tvm"</code> declarations for every TVM opcode that the stdlib surface wraps, available to <code>unsafe</code> code only

Module-path collision rules:

- third-party libraries may not publish into the `std::*` namespace;
- contract code may publish its own `mod foo::bar` tree freely;

- the `use` resolver searches the current crate first, then `std::*` as a reserved root, then external dependencies listed in the build manifest;
- `use std::prelude::*`; is applied implicitly at the top of every source file under edition 0.5.0.

A full item-level stdlib reference (every exported signature with documentation) is scheduled as a sibling document, `doc/func_v0.5.0-stdlib-ref.md`, to be written alongside the reference compiler implementation. The tree above is the frozen surface; the signatures inside each module may still evolve up to edition freeze.